

Distributed Computing

BigData I : Hadoop

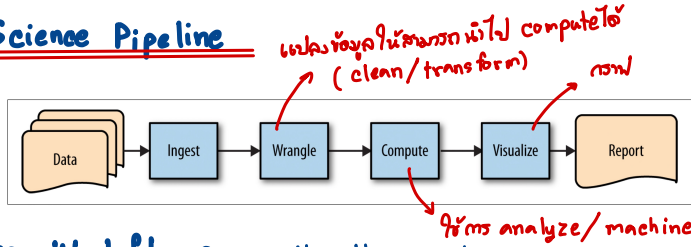
- Data แบ่งออกเป็น 3 ประเภท
 - Structured data เช่น ข้อมูลที่เป็นตาราง (ข้อมูลใน shopee 1 คิว = 1 log)
 - Unstructured data เช่น text, log file, ภาพ, เสียง, วิดีโอ, graph data
 - Semi-Structure data เช่น HTML file, XML, JSON (facebook, web)
- Bigdata Characteristics → **5Vs**

- Volume** ข้อมูลมีขนาดใหญ่มาก
- Variety** ข้อมูลมาจากหลายแหล่ง และหลาย type ทั้ง structured และ Unstructure
- Velocity** ข้อมูลไหลเข้ามาจำนวนมาก และตลอดเวลา เช่น real time data (data จาก sensor ต่างๆ)
- Veracity** คุณภาพของข้อมูล อาจมี biases, noise, และ ไม่ชัดเจน
- Value** สามารถนำมามีประโยชน์ แล้วสร้างคุณค่าได้

- Data Product
 - A data driven application เป็น software ที่ใช้ data ในการสนับสนุน
 - A system that learn from data เป็นระบบ software/hardware ที่ปรับตัวได้ และใช้ประโยชน์จากข้อมูล
 - หรือ data product มาช่วยในการตัดสินใจทางธุรกิจ

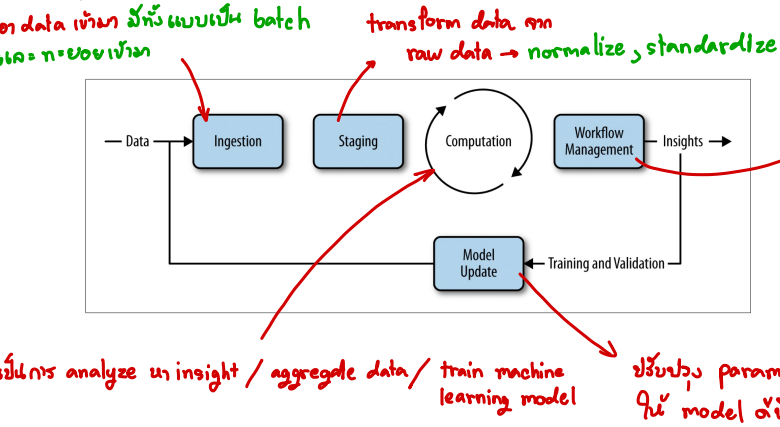
- Data Science วิชาที่ผสมผสาน disciplines ในการสร้าง data product ที่มีประสิทธิภาพ
 - 1.) Machine Learning 2.) Software Engineering 3.) Statistics 4.) Database Management 5.) Distributed Computing

Data Science Pipeline



- workflow สำหรับการสร้าง data product
- มักใช้ภาษาเขียนส่วนมากคือ scripting languages
- ไม่เหมาะกับ big data / real time processing ***
- ต้องการเครื่องมือที่ช่วย automate ในขั้นตอนด้วย

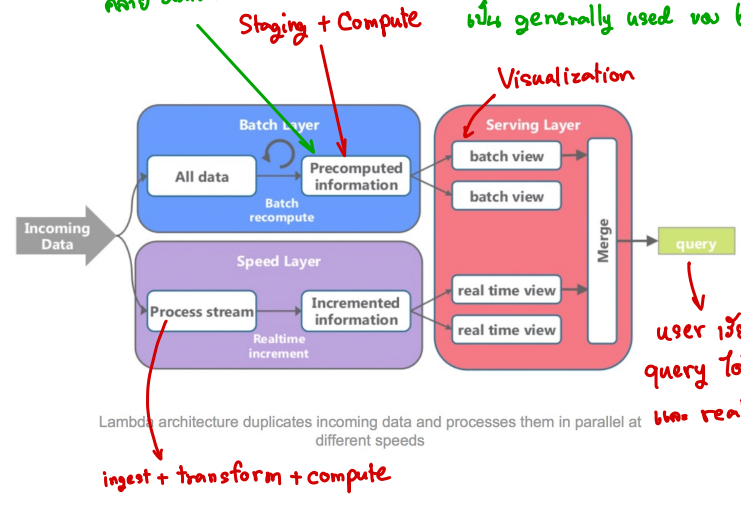
BigData Workflows → iterative model



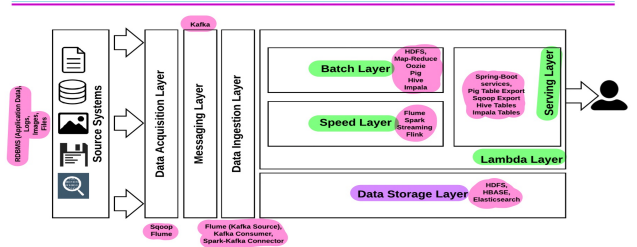
- เข้ามาแทน pipeline แบบเดิม เพื่อ scalability และ automation
- ทำในขั้น automation ไม่มีความจำเป็นที่ต้องเขียน
- เป็นลักษณะ workflow ที่ต้องมีการทำงานอยู่เรื่อยๆ
- user ไม่จำเป็นต้อง workflow management ด้วย scripting language เพื่อใช้ในการทำงานอัตโนมัติตั้งแต่ต้นจนจบ

Lambda Architecture → data-processing architecture ที่รองรับทั้ง batch และ stream (real time) processing

คือ datawarehouse ใน BI tools ที่ใช้สำหรับ handle ได้ batch ในอีกด้านด้วย รองรับ real-time monitoring ได้ เป็น generally used var big data architecture

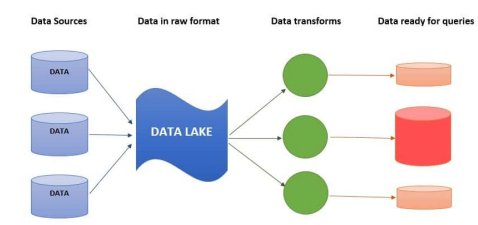


- ประกอบด้วย 3 ส่วน
- Batch layer** เก็บ raw data และ pre-compute batch data
 - Speed layer** รับข้อมูลที่เร็ว (real time) ด้วย low latency ข้อมูลเป็นก้อนเล็กๆ มาเป็น package
 - Serving layer** เป็น user interface ในการดึงข้อมูล



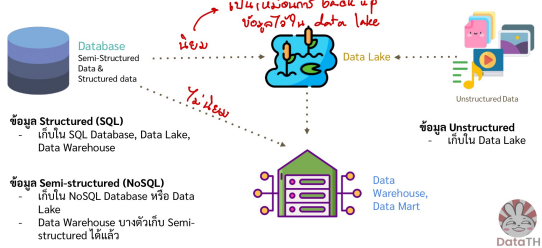
Data Lake

→ เป็นที่จัดเก็บ data ในรูป raw data เมื่อ user จะใช้ข้อมูลไป process ทำ task ที่ต้องการ
ประกอบด้วยการ reporting, visualization, analytics และ machine learning



- Data lake
- structure ข้อมูลเป็นตาราง
 - semi-structure CSV, logs, XML, JSON
 - unstructure email, documents, PDF
 - binary image, audio, video
- } centralize data store ที่รองรับ data ทุกๆรูปแบบ

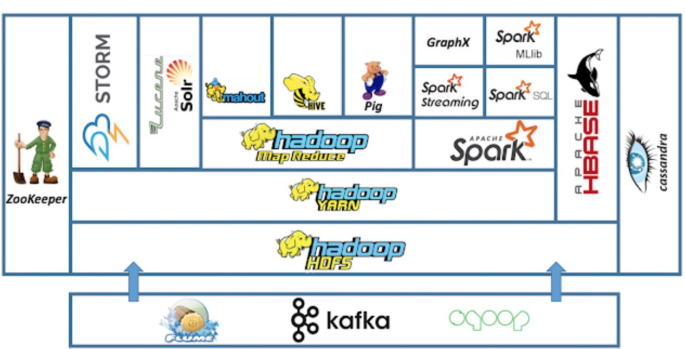
Pattern พื้นฐานในการเก็บข้อมูลแบบ Big Data



- Structure และ Unstructure data ที่ไหลเข้า data lake สามารถ implement ได้ด้วย distributed data storage เช่น Hadoop File System, Amazon S3
- เมื่อ data ถูกเก็บใน data lake จะใช้กรณี ETL หรือ Extract - Transform - Load เพื่อส่ง data warehouse หรือ local datamarts เพื่อนำมาใช้ Hbase หรือ Hive ทำการวิเคราะห์ analyze ด้วย analytic tool หรือ program ต่างๆได้

Hadoop

→ เป็น ecosystem ของเครื่องมือ ที่ใช้ประมวลผลข้อมูลขนาดใหญ่ Bigdata Pipeline (OS for Bigdata dev. by Google) 2009

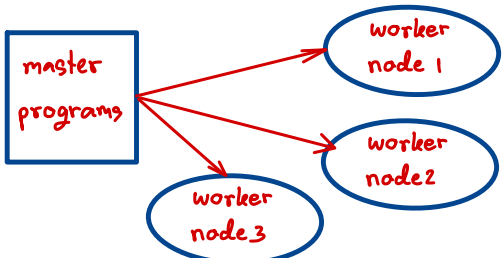


- Sqoop, Flume, Kafka design มาสำหรับรองรับ data ingestion เพื่อทำการ import data เข้ามาใน hadoop file system หรือ DB
- Hbase, Hive คือเป็นตัว data management
- Spark's GraphX, MLlib, Mahout คือเป็น analytical pack. สำหรับ bigdata
- คือกระชับ การประมวลผล dataset ขนาดใหญ่ ไปใน คอมพิวเตอร์เครื่อง เพื่อทำการคำนวณ/ประมวลผลของทั้งระบบเร็วขึ้น
- hadoop คือ require distributed algorithms เพื่อควบคุมเครื่องคอมพิวเตอร์ให้เป็น cluster และ networking services

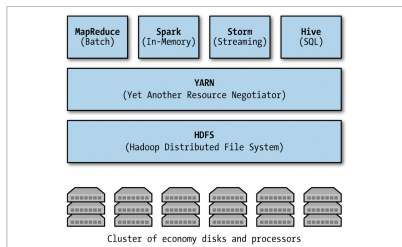
* Server ส่วนใหญ่จะ Host OS เป็น linux แล้วใช้ hadoop มา on top

Basic Concept

- distributed systems ต้องมี requirement ดังนี้
 - 1.) Fault Tolerance → เมื่อเกิด error fail ในส่วนใดส่วนหนึ่ง ก็ไม่ fail ไปด้วยทั้งหมด แต่จะเป็น lower performance state เมื่อถึงที่ fail recovery แล้วจึง join ใ้
 - 2.) Recoverability → ระบบ fail แล้วจะกลับมาทำงาน
 - 3.) Consistency → fail one job ไม่กระทบ final results
 - 4.) Scalability → สามารถเพิ่ม resource ได้ เพื่อเพิ่ม performance ของระบบ
- hadoop คือเข้ามา address requirement ของ distributed system ด้วย concepts ดังนี้
 - 1.) data จะถูก distribute กันทั่ว เพื่อ add เข้า cluster ของเครื่อง และ ถูกเก็บในหลายๆ node
 - 2.) แต่ละ node จะ process data ในส่วนนั้นๆ และ ส่งผล traffic ไปมาสื่อสารกัน
 - 3.) data จะเก็บใน block ที่ fixed size (usually 128 MB) แต่ละ block จะ duplicate เก็บหลายๆเครื่อง เพื่อ provide redundant และ data จะสามารถลบออกได้ ไม่ยุ่งยาก → ข้อเสียคือ เปลือง resource เพราะต้องสร้างหลาย copy
 - 4.) 1 job จะแตกออกเป็นหลายๆ task แต่ละ node ก็จะทำ task ใน 1 block ของ data
 - 5.) การทำ job จะใช้แค่ high level programming ลดความซับซ้อนในการ dev. ระบบ
 - 6.) minimized traffic ระหว่าง node ในระบบการกระจายแต่ละ task แต่ละ node จะทำงานของตัวเองคือจะไม่มีการคุยระหว่าง node ป้องกันการ dead lock เป็นช่วงที่ Server ทำงานตัวส่วนอื่น
 - 7.) Run job ด้วยกันหลายๆเครื่อง เพื่อเป็น fault tolerant (ปกติไม่ถ่วงกัน → กิน resource)
 - 8.) จะมี master programs ทำหน้าที่คอยให้ worker nodes แล้วทำงานกันแบบ parallel



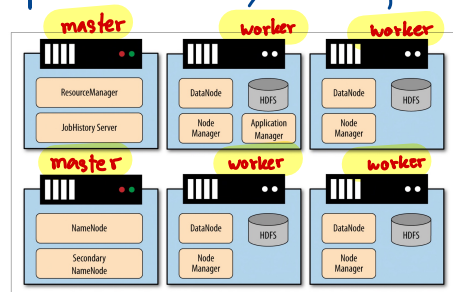
Hadoop Architecture



→ จะประกอบด้วยส่วนหลัก 2 ส่วน

- 1.) **HDFS** เป็นระบบจัดเก็บ file ภายในระบบ cluster ของ hadoop สามารถ access กับ cluster ได้
- 2.) **YARN (Yet Another Resource Negotiator)** เป็น cluster resource manager เป็นตัวจัดการ
 การกระจาย task กระจายไปทั่วทั้ง cluster (scheduler)
 - HDFS และ YARN จะทำงานร่วมกัน เพื่อลด network traffic (data block อยู่ไหน ในทุก node ใน node นั้น)
 - 1 copy data ไว้เป็น fault tolerant และ recovery ได้ เมื่อต้อง maintain consistency ของ data
 ถ้า data copy ไว้ 3 block เวลา update ก็ต้อง update ทุก block

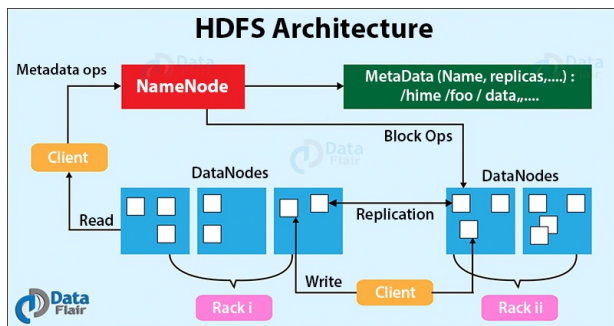
- Speculative Execution (ทำงานตามตารางเวลาที่กำหนด) คือการ duplicate task เพื่อเป็น redundancy เช่น ถ้า task หนึ่งทำงานช้าผิดปกติ
 อีก node ก็ทำการ task เดิมซ้ำกันขึ้นอีก อันไหนเสร็จก่อนก็จะลบของอีกอันออก
- YARN และ HDFS จะทำงาน daemon process จะเป็นการรับ input ใน output แล้วกระจาย input ต่อไป ทำไปเรื่อยๆ จนกว่าจะหยุดการทำงาน
- set ของ machine ที่ run บน HDFS และ YARN แต่ละ cluster จะมีชื่อ nodes
- แต่ละ node จะมี 2 ส่วน
 - 1.) **master node** จะทำหน้าที่ coordinate service ของแต่ละ worker node
 - 2.) **worker node** จะรับ task จาก master node มาทำ



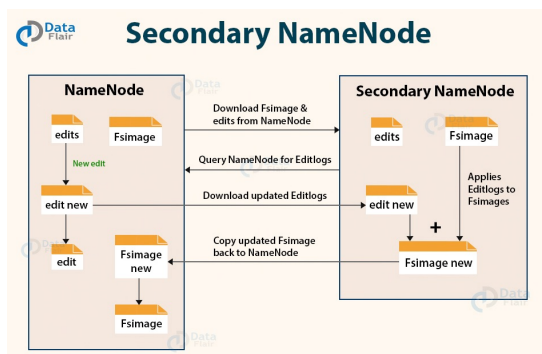
HDFS

จะประกอบด้วย master และ worker nodes ประกอบด้วย

- 1.) **NameNode (Master)** จะทำหน้าที่เก็บ directory tree ของ file system, file metadata และ location ของแต่ละ file ใน cluster
 client จะ request ข้อมูลเกี่ยวกับ file ในแต่ละ node ไปจาก NameNode แบ่งเป็น
 - **fsimage file** เป็น file ที่เก็บข้อมูลระบบ
 - **edit log file** เป็น file ที่เก็บ change ที่เกิดขึ้นในระบบ file แล้วจาก NameNode ทำงาน

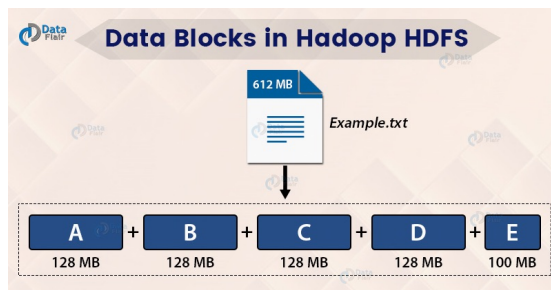


- 2.) **Secondary NameNode (Master)** จะทำ housekeeping task และ check pointing บน server ของ NameNode
 เพื่อลดภาระของ NameNode เช่น ไปอ่าน edit log file ใน NameNode แล้วมาเขียนลง fsimage ของ server ที่ copy มา แล้วเอา fsimage
 ที่ update แล้วอัปเดตขึ้น NameNode

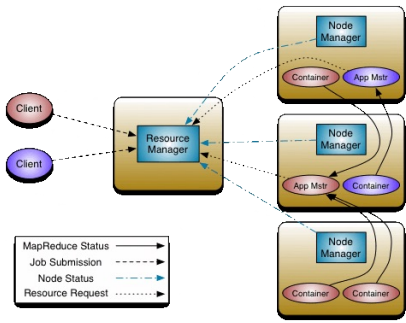


* สำคัญไม่ stand by NameNode เพราะ
 ถ้า NameNode มีปัญหา stand by
 จะทำงานแทน

- 3.) **Data Nodes (Worker)** จะทำหน้าที่เก็บ และ manage HDFS block บน local disk และ report status กลับไปให้ NameNode
 เพื่อบอกว่าทำงานอยู่ปกติ หรือไม่ปกติ



YARN



- 1.) Resource Manager (Master) เป็นตัวบริหารจัดการ resource แยกออกจาก client เช่น memory, processor core ใช้ไปกับ app ในหน่วย scheduling job และกระจายงานไปทั่วทั้ง cluster ในหน่วย Node
 - 2.) Application Master จะถูกสร้างขึ้นต่อ 1 application จะเป็นตัว coordinate application ที่ run อยู่ใน cluster และจะคอย schedule Task Resource Manager
 - 3.) Node Manager (Worker) จะเป็นตัว run และจัดการ task ในหน่วย node และ report status กลับมาหา Resource Manager และคอยจัดการ container ใน node
- * container คือ package ที่ใช้ทรัพยากรในการคำนวณ เช่น RAM, CPU, Network
- * Master process ไม่สามารถรันอยู่ใน node เดียวกัน จะทำใน 2 node หรือถ้าเป็น cluster ใหญ่ มี 2 master พอ

HDFS Concepts

- ระบบไฟล์ที่กระจายข้อมูลซ้ำกันหลายครั้ง ไม่เหมือนกับ file ทั่วๆไป 1 file = 1 block คือทำซ้ำข้อมูล block ทั่วทั้ง cluster
- ใช้ pattern write once, read many ไม่สามารถ write หรือ append file ได้ จึงไม่เหมาะกับ real-time data
- Block หรือ file สามารถกระจายไปหลาย node เพื่อ fault tolerance

Working with HDFS

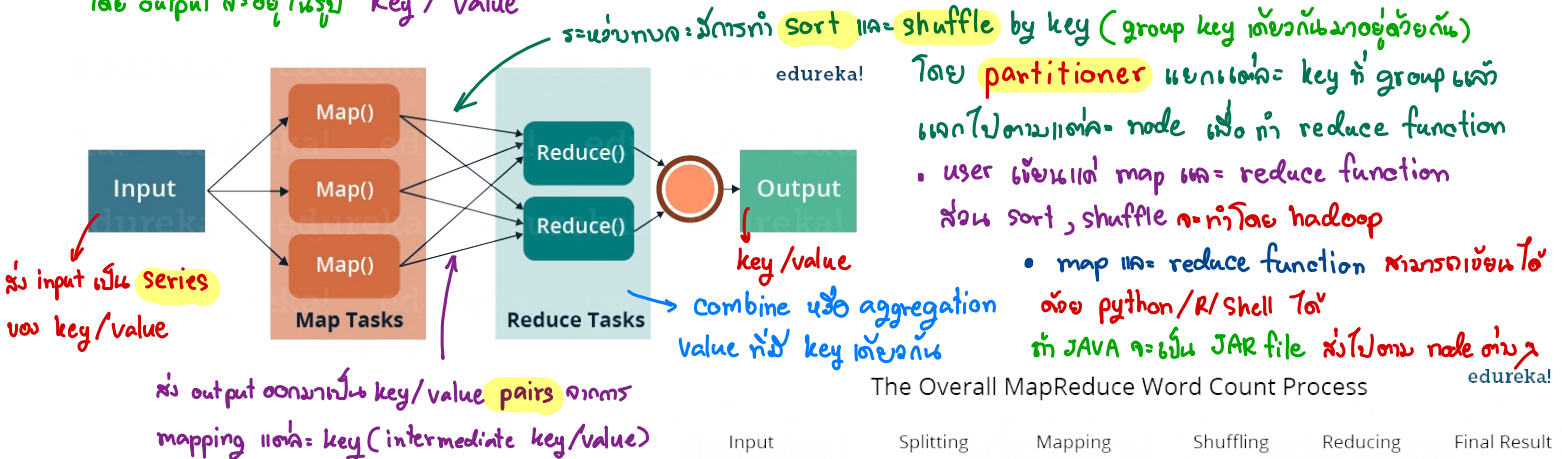
1.) Command Line interface (CLI) / 2.) HTTP interface (web browser) / 3.) API (programming interface written in JAVA)

```
$ hadoop fs -put x.txt /user/cloudera/y.txt
$ hadoop fs -get /user/cloudera/y.txt x.txt
$ hadoop fs -mkdir -p /user/cloudera/corpora
$ hadoop fs -ls /user/cloudera
$ hadoop fs -mv /user/cloudera/x.txt /user/cloudera/rawdata
$ hadoop fs -cp /user/cloudera/x.txt y.txt
$ hadoop fs -rm /user/cloudera/x.txt
```

Annotations: put local file to hadoop, get from hadoop, mkdir -p create directory, ls list contents of file, mv move file, cp copy file, rm remove file, chmod 664 change mode, (-rw-rw-r- permission is set for the given file), read, write, execute.

Map Reduce

- เป็น computation part ใน data pipeline
- เป็น framework ในการ compute ที่ออกแบบมาเพื่อ distributed computation ที่ fault-tolerant ทั่ว cluster ทั่ว
- ใช้ functional programming ในการทำ task ที่แยกกันไว้ เช่น node หนึ่งทำ min-max อีก node aggregate data ทำตาม function เป็น task ที่กระจาย data ใน node ทำตาม task หรือกระจาย node ใน cluster ทั่ว
- data จะถูกส่งมาที่ node แล้ว function จะทำ output ของ function ไปเป็น input ของอีก function หนึ่ง
- Function จะส่ง output มาเป็น input ของอีก function หนึ่ง ไม่ส่ง state หรือค่าที่เก็บไว้ในหน่วยประมวลผล
- ประกอบด้วย 2 function distributed work → map, aggregate data → reduce
- การทำงานของ map และ reduce คือ map ทำหน้าที่ map data และ reduce ทำหน้าที่ aggregate data

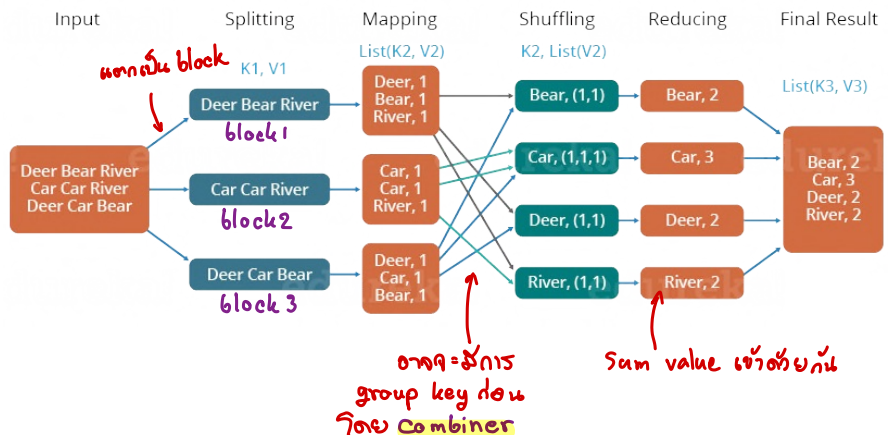


Word Counting problem (Pseudo code)

```
def map (dockey, line);
    for word in line.split();
        emit(word, 1);
def reduce (word, values);
    count = sum (value for value in values);
    emit(word, count);
```

Annotations: dockey, line, word, line, separator, word, 1, key/value pair, value for value in values, word, count.

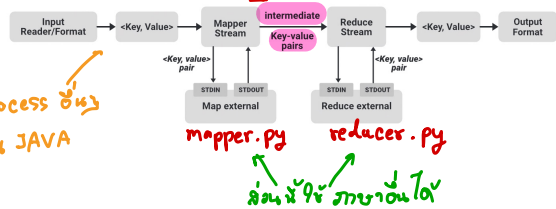
(note: emit – perform Hadoop I/O which sends its arguments to the next phase of MapReduce pipeline)



Hadoop Streaming

- สามารถใช้ Shell/R/Python ในการรัน MapReduce jobs ได้
- hadoop streaming จะเป็น concept จาก standard Unix Streams
- ทุก task ที่รันจะรันมาทำงานตามคำสั่ง assign standard file
- input ของ map และ reduce function จะ read ผ่าน stdin file
- output ที่ได้จากทั้ง 2 function จะ write ไปที่ stdout file
- โดยออกมาเป็น key/value pairs (separated by \t) หรือ tab
- error จะเขียนไปที่ standard error
- ทุก process ของ hadoop streaming, map reduce จะทำงานบน file

Hadoop Streaming



Combiner (optional function) → ใช้ combine key และ value โดยรวม group by key

- ทำหน้าที่รวม key/value ที่ได้จาก output ของ mapper task จำนวนน้อยลง ทำใน shuffle & sort ใช้เวลาลดลง และ ลด network traffic (mapper - local reduction)

Ex: Mapper 1 output → k1:1, k2:1, k3:1, k1:1
 Mapper 2 output → k1:1, k3:1, k3:1
 Mapper 3 output → k1:1, k2:1, k2:1

combiner → (k1:1,1), (k2:1), (k3:1)
 (k1:1), (k3:1,1)
 (k1:1), (k2:1,1)

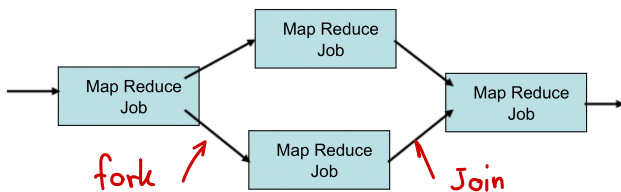
shuffle & sort → k1:1,1,1,1
 k2:1,1,1
 k3:1,1,1

* ถ้า task / data ไม่เยอะมาก ไม่จำเป็นต้องใช้ combiner ก็ได้

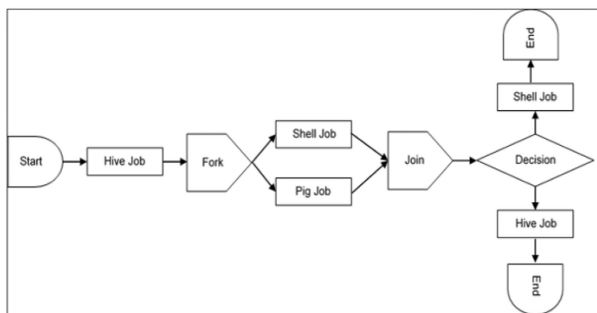
Partitioner → หน้าที่แบ่งว่า key/value แต่ละตัวจะส่งไป reducer ตัวไหนบ้าง

- สามารถเขียน partitioner เพื่อปรับสมดุลได้
 - ถ้า default ก็คือ แบ่งเท่าๆกัน ตามจำนวน key ที่ได้ เพื่อ balance load (hash partitioner)
 - แต่ละตัวจะส่งไปให้ mapper รับแล้วรันตามคำสั่ง key → ผ่าน hash function จะได้จำนวน id ของ key เพื่อส่งไป reducer
 - hash function คือ เอาค่า key ที่เป็น string แปลงเป็นตัวเลข แล้ว modulo ด้วยจำนวน reducer ที่เราอยากได้
- key(str) → key(int) → $n_{key} \% n_{reducer}$ → เลข 1 → reducer 1
 ...
 n → " — n
- ถ้า key = แบ่งเท่ากัน แต่บาง key value ไม่เท่ากัน ก็คือ key imbalance → อาจต้อง custom partition ของ

Job chaining → เมื่อเราอยากทำงานที่ซับซ้อน เราสามารถแตกออกมาเป็นหลาย MapReduce Job แล้วทำงานต่อกัน



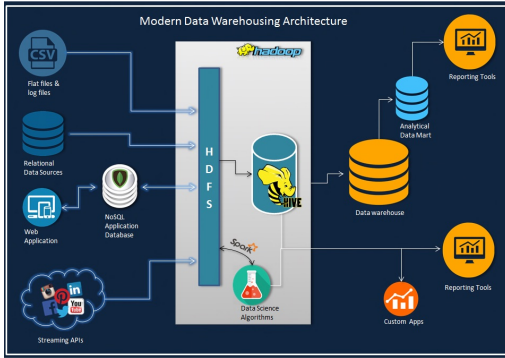
- อาจจะทำกันในรูปแบบ เส้นตรงต่อกัน หรือ แยกทำงานแบบขนาน
- ตัว workflow อาจจะไม่ใช้แค่ Mapreduce Job สามารถผสมผสานกับ การเขียนโปรแกรมธรรมดาได้
- workflow จะอยู่ในรูปแบบ directed acyclic graph (DAG)



- ใช้ apache Oozie เป็นตัว scheduler system ในการ manage เพื่อ automate workflow ใน hadoop env.
- สามารถ รันงานที่ซับซ้อนต่อกันแบบ sequential หรือ parallel ได้ ด้วย fork & Join construct
- Oozie ใช้ได้หลาย hadoop job เช่น hive, Pig, Sqoop รวมไปถึง JAVA, Shell script
- Node ใน workflow สามารถเป็นได้ทั้ง
 - 1.) Control flow node → start, end, kill, fork, join, decision
 - 2.) Action node → Map-Reduce jobs, Pig jobs, Shell Command

- สามารถสร้าง workflow ผ่าน Oozie Gui editor จะได้ output เป็น xml file

Bigdata 2 : Hive → เป็น data warehouse framework ที่ built on top hadoop



- เราจะสามารถใช้งานได้ทั้ง SQL interface
- โดย data จะเป็น schema และเก็บอยู่ใน HDFS
- Hive คือภาษา SQL ของฮอว์ดิว เรียกว่า Hive Query Language หรือ HQL
- ใช้งาน Structure data
- Hive Query Language (HQL) คือ support คำสั่ง SQL หลักๆ ที่ใช้งานบ่อยๆ
 - Data Definition Statement (DDL) : CREATE Database / Schema / table
 - Data Manipulation Statement (DML) : INSERT, UPDATE, LOAD
 - Data Retrieval : SELECT
- สามารถเขียน function เองได้ทั้ง JAVA, Python etc... by Hadoop Streaming

- **Hive command** เป็นคำสั่งในการสั่ง Hive } คือถูก compiled เป็น **HDFS operation** และ/หรือ **Map Reduce Job**
- **HQL queries** เป็นคำสั่งในการ query ข้อมูล } เช่น **SELECT, aggregate**
- Hive นั้นจะส่ง ข้อจำกัด ของ HDFS คือ **WORM (write once read many)** ทำให้ไม่สามารถ **update** ได้โดยตรง
- แต่สามารถทำ **INSERT, UPDATE, DELETE** ได้ แต่ไม่รองรับการลบ และสิ่งที่ได้จากการทำคือ **small delta files** (น้อยๆ log rec.)
- **Delta files** คือถูก merge กลับเข้ามา ใน database ด้วย **map reduce job** ที่ run background ด้วย **Meta store server**
- Hive จะสามารถทำ **indexing** เพื่อเพิ่มความเร็วในการ query (ปกติจะเป็นการ full scan table ทุกครั้ง ถ้าไม่ทำ)
- Hive จะเก็บ **meta data** ของ database (properties ของ data เช่น schema, ท็อป, คอมพริส, ...) ใน **MetaStore**
- ส่วนใหญ่จะใช้ **MySQL database server** (อยู่บน HDFS) เพื่อความเร็วในการ update และด้วยข้อมูล meta data จะมีการเปลี่ยนแปลงบ่อยกว่า data
- สามารถทำ query large dataset แบบ **batch**

- Create database

- Create a database
hive > create database log_data;

```
hive > create database if not exists log_data;
( create database if not exists )
```

- เมื่อสร้าง database ใน Hive น. ชื่อ schema ที่เก็บอยู่ใน Metastore ถ้ามี database อยู่แล้ว น. นั้น error

- Show names of all existing databases
hive > show databases;
- Set a working database
hive > use log_data;
- Show tables in the working database
hive > show tables;
- Show the schema of a table
hive > describe test;
- Add a field in the schema of a table
hive > alter table test add columns (address string);
- Delete a table
hive > drop table test;

- **For a managed table**
Hive > create table memo (lineno string, linetext string) row format **delimited fields terminated by '\t' stored as textfile**;
Row format is used to indicate that the data file (in HDFS) has '\t' character as delimiter between fields.
Note : A table can be stored as **ORC** (optimized columnar format)

- **For an external table**
Hive > create external table memo (lineno string, linetext string) row format **delimited fields terminated by '\t' stored as textfile location /user/cloudera/external_table**;

- Create a table

- By default, เมื่อสร้าง table ใน Hive จะ Hive จะทำการ บันทึกข้อมูลไว้ใน warehouse directory
- เมื่อต้องการจะสร้าง external table non warehouse directory ได้
- ถ้าเรา drop external table (delete) data ในส่วนนั้นจะไม่ถูกลบออกหรือจะยังอยู่ แต่จะลบเฉพาะ meta data (ไม่ไปลบข้อมูลจริง)
 - อยู่ ใน directory ของ user

- Transform data to table

Hive provides a way to apply a regex (regular expression) to known record formats to deserialize or parse each row into its constituent fields.

```
hive> create table apache_log (
    host string,
    identity string,
    user string,
    time string,
```

✍ 98 regex ใช้แทน data ที่ 100 มีนัย
เพื่อแทน 99 ธรรมดา

```
request string,  
status string,  
size string,  
referer string,  
agent string )
```

row format **serde**

'org.apache.hadoop.hive.serde2.RegSerde' with Serdeproperties {"input.regex"="(^[^"]*)(\"[^\"]*\")(-|\\[[^\\[\\]]*\\])\"([^\"]*)(-|[-0-9]*)(-|[-0-9]*)(?:([^\"]*\"[^\"]*\")([^\"]*\"[^\"]*\")*)\"([^\"]*)(-|\"[^\"]*\")\""}; "output.format.string"="%1\$s %2\$s %3\$s %4\$s %5\$s %6\$s %7\$s %8\$s %9\$s" stored as textfile;

- Serde (Serializer/Deserializer) along with regular expression can be used to specify record format of the input file, which may not be in Hive format, so the content of the input file can be loaded into a Hive table.

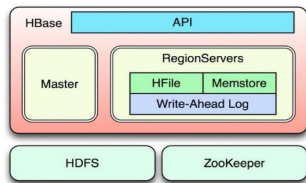
- **SQL Regular Expression** **ואם** **Serialize data** **ואם** **column names**

- Loading data

- when load data into table Hive will not query show read schema as file structure but in schema you can see missing field with data type which will return as Null
- we load data into table through 2 ways 1.) 9th bulk load data command 2.) 9th insert query and then 10th insert command

Big Data 3 : Hbase

HBase Architecture



- Hbase คือ ระบบข้อมูลแบบ NoSQL ซึ่งจะเป็น column-family หรือ column-oriented db
- NoSQL ก็เป็น non-relational database ใช้จัดเก็บข้อมูลที่ไม่ใช่โครงสร้างแน่นอน (Unstructure data) ซึ่งสามารถออกแบบให้เก็บข้อมูลได้หลายแบบ เช่น graph database, document database, key/value datastore แล = column-family database
- Column-family / Column-Oriented คือ การเก็บข้อมูลแบบ column by column ข้อมูลจะถูกใส่ตาม column เป็นหลัก บาง row อาจจะไม่ใส่บาง column (flexible schema) แปลว่าการเก็บข้อมูลจะเป็น row-oriented

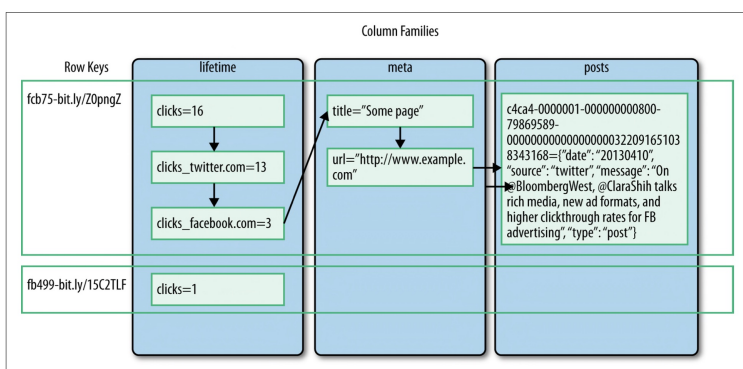
- ซึ่งแตกต่างจาก Hive ที่จะเป็นการรัน SQL-based กับ structure data ที่เก็บใน HDFS
- โดย Hive จะใช้ HDFS แล = MapReduce มา process data ใน batch mode
- คุณสมบัติ WORM ไม่ได้ออกแบบให้ support random real-time low latency read/write แล = application บางอย่างต้องรัน โครงสร้างข้อมูลกันมานานหลาย ปีแล้ว = record จึงไม่เหมาะกับการรัน Hive ทำใน Hbase เข้ามาตอบโต้กับระบบหลักแทน

Table 6-3. Unstructured events

Event ID	Event Timestamp	Event Type	User ID	Post ID	Comment	Receiver User ID
1	1370139285	Like	jjones	521		
2	1370139285	Share	smith	237	This is hilarious!	342
3	1370139285	Share	emiller	963	Great article	

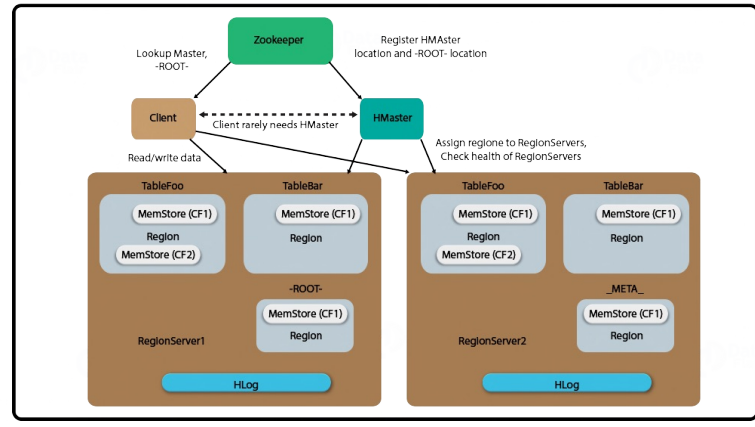
- Hbase พัฒนามาจาก Google's Bigtable โดยจะมี feature หลัก 3 อย่าง
 - 1.) Random (row-level) read/write access คือไม่ถูกจำกัดด้วย WORM คือ read/write row ใดก็ได้ แต่ทำทั้ง row
 - 2.) Strong Consistency ทำในเวลาที่ read/write ได้ real-time เวลาหลาย user ทำพร้อมกันคือจะคอยเช็คกัน update ที่สุด
 - 3.) Schema-less or flexible data modeling แต่คือ row มีโครงสร้างไม่แน่นอน คือเพิ่ม/ลด column ในแต่ละ row ได้
- Hbase คือจัดเก็บ data ตาม row ในแต่ละ row คือถูกกำหนดด้วย Unique row key (เหมือน Primary key ของ row)
- Row key คือไม่มี datatype แล = เก็บในรูปแบบ byte arrays
- ดังนั้น row key คือเป็นได้ทั้ง string หรือ binary บางทีอาจจะเป็นตัวอักษร (Serialize data structures) key ของแต่ละ row คือมีค่าเฉพาะของ byte ไม่เหมือนกัน
- ส่วน Table row คือ Sorted ตาม row key เรียงตามแต่ละ byte อัน byte ใดมาก่อน
- data ในแต่ละ row คือถูก group เป็น column-family ภายใต้อัน column-family คือประกอบด้วยหลาย column ที่ relate กัน
- table คือถูกแบ่งออกเป็น regions หรือ set ของ row ที่คือ manage ด้วย region server
- ข้อมูลใน region คือถูกเก็บใน Hfiles (ข้อมูลในรูปแบบของ key/value pair) บน HDFS

PERSON TABLE					
row key	personal_data		demographic		...
PersonID	Name	Address	BirthDate	Gender	...
1	H. Houdini	Budapest, Hungary	1926-10-31	M	
2	D. Copper	New Jersey, USA	1956-09-16	M	
3	Merlin	Stonehenge, England	1136-12-03	F	
...	...	...	...	...	
500,000,000	F. Cadillac	Nevada, USA	1964-01-07	M	



key	t1			
	t2	t3	t4	t5
1	ca1-t3	cf1-colB	cf2-colC	cd1-t3
2	ca2-t3	cb2-t3		cd2-t3
3	ca3-t3		cc3-t3	

HBase Architecture



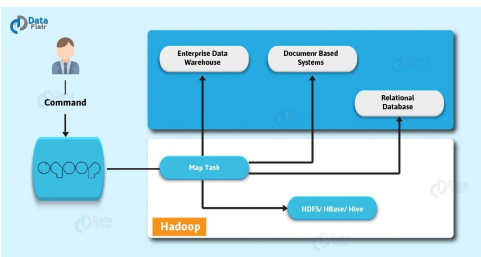
ในตัวของ Hbase มี 3 ส่วนหลักๆ ที่ทำงานร่วมกัน

- **Master responsible**
 - ทำหน้าที่ **coordinate region server** กำหนดว่า region ให้อยู่ที่ server ไหน และ re-assigning ได้ เพื่อ recovery ในกรณีที่โหนดมีปัญหา หรือมีการทำ load balancing ของ table region
 - คอย **monitor region server** ทั้งหมดโดยจะ report หา **ZooKeeper**
 - เป็นตัวเชื่อมกับทำหน้าที่ **admin function** เช่น **create, delete, update table** เพราะ master จะรู้ว่า data อยู่ไหน
- **Region Server** → มี run อยู่บน data node ของ HDFS มี 4 ส่วนประกอบ
 - 1.) **WAL : Write Ahead Log** → เป็น file ที่อยู่บน HDFS WAL คือการเขียนข้อมูลก่อน new data ใน memstore ที่ยังไม่ได้อัดลง write ลง Hfile เพื่อในกรณีที่ server restart ได้เมื่อ region server เกิด fail
 - 2.) **Block Cache** → คือ **read cache** ใช้เก็บ data ที่ถูก read บ่อยๆ ใน memory เมื่อ memory เต็ม ก็จะลบทิ้งทิ้งที่เก่าที่สุดออกไป
 - 3.) **Memstore** → คือ **write cache** ใช้เก็บ และ buffer new data ที่ยังไม่ได้อัดลง disk (1 memstore / 1 column-family / 1 region) เมื่อ data ใน memstore เต็มก็จะถูก sorted และ write ลง Hfile โดยจะเรียงตาม Hfile ชื่อ 1 column-family
 - 4.) **Hfile** → เป็น file ที่เก็บ **Key/Value** ที่ถูก sort ตาม key บน disk
- **Zookeeper** → เป็นตัว coordinator
 - Hbase จะใช้ Zookeeper ในการ maintain server state เพื่อ monitor สถานะของ server แต่ละตัว แล้วส่งข้อมูลมา เป็นระยะๆ (เพื่อลดภาระของ Master)
 - เป็นตัวเก็บ **Meta table** ซึ่งจะมีเก็บ Meta data ซึ่งจะมีเก็บ location ของทุก region ในแต่ละ cluster เมื่อตัว Hbase อยากรู้ว่า region ให้อยู่ที่ไหน ก็จะมาถามที่ Zookeeper เพื่อค้นหาตำแหน่งของ Meta table

Bigdata 4 : Data Ingestion

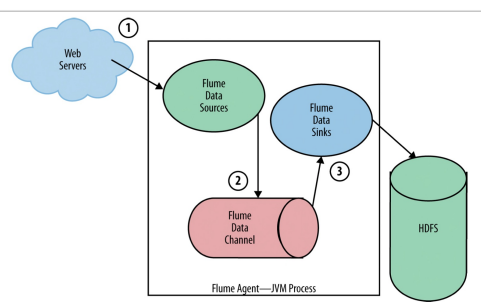
- Data ingestion คือ process ในการ import external data เข้าสู่ฐานข้อมูล สำหรับทำการวิเคราะห์ข้อมูล เป็นขั้นตอนแรกของ data science pipeline
- เรา import data สามารถได้ทั้ง structured data, semi-structured/unstructured data หรือ streaming data ที่มาจาก log file หรือ external event

• Sqoop → เป็น tool สำหรับ import structured data (table based) จากภายนอก เช่นจาก MySQL, Oracle



- Sqoop สามารถใช้ร่วมกับ continuous ingestion (Streaming data)
- ตัว sqoop จะเข้าไปอ่าน schema ของ RDBMS โดยตรง แล้วใช้ Map Reduce ในการ import และ export data ใน Hadoop
- ใช้ Map Reduce Job (ใช้ map-only) ในการดึงข้อมูล Database ภายนอกเข้ามาใส่ HDFS, Hive หรือ Hbase โดย based on metadata ของ table นั้น
- ในการนำ table เข้ามา HDFS ก็จะมีรูปแบบที่จำเพาะคือ import เข้ามาเป็น file แบบไหนสามารถเป็นได้ทั้ง text file, binary format (Avro) หรือ sequence File (key/value pairs)

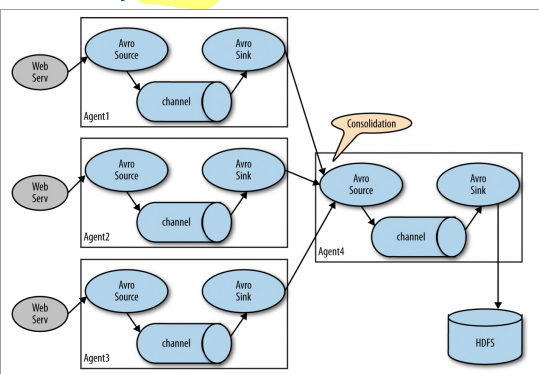
• Flume → ออกแบบมาสำหรับ ingest streaming data ที่มี high volume จากหลาย source เข้าสู่ Hadoop



- flume จะ import data ในลักษณะของ log file จากหลาย web server
- log file จะเป็น file ที่เก็บ log record ที่มีการโต้ตอบข้อมูลต่อกันมาเรื่อยๆ update ตลอด
- flume สามารถรับข้อมูลที่มีปริมาณมากได้ เช่น network traffic data, social media
- ถูกออกแบบมาให้มีความทนทานต่อ fault - tolerance และ scalability สามารถทำงานแบบ distributed
- 1 unit ของ event (1 log statement) ที่เข้ามาจะถือเป็น 1 data flow ที่ต่อเนื่องผ่านแต่ละ stage จากต้นทางไปถึงปลายทางด้วย sequence of hops
- โดยตัว Flume agent จึงเป็น unit ตัวใน Flume data flow ที่รับ flow data เข้ามา

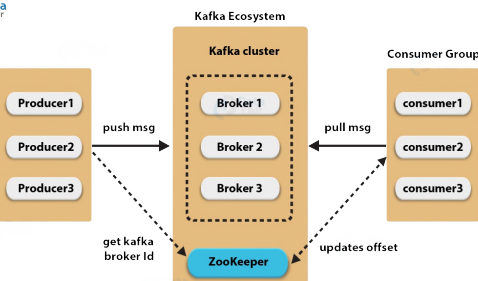
• Flume Agent จะมี 3 ส่วนประกอบ

- 1.) Source → เป็นตัวรับข้อมูลจาก Server ภายนอก/agent อื่นๆเข้ามา (read จาก log ด้วย Unix command)
- 2.) Channel → เป็นตัวพัก data ในการที่ agent ตัวอื่นส่งข้อมูลมา และจะส่งข้อมูลต่อไปยัง sink เป็นแบบ batch
- 3.) Sink → เป็นตัวส่งข้อมูล data ไป agent ตัวต่อไปหรือพร้อมรับ หรือ write ลง HDFS เมื่อถึงจุด process ที่เหมาะสม



- เมื่อมีมากกว่า 1 agent Source และ Sink ของ agent ที่ทำงานเหมือนกัน จะติดต่อกันด้วย protocol ที่เรียกว่า Avro RPC (Remote Procedure Call) ในการแลกเปลี่ยน schema
 - Avro เป็น data serialization ใน Hadoop คือแปลง data เช่น memory buffer, data structure หรือ object state ให้กลายเป็น binary form คือการทำให้เป็น byte เรียงต่อเนื่อง แล้วส่งต่อไปในรูป Avro data file
 - สำหรับ Avro data file นั้นตัว Avro จะเก็บ serialize data และ schema ในรูปแบบ JSON format (Javascript Object Notation)
- เมื่อส่งไป agent ปลายทางจะทำการ deserialize data ออกมาเป็นแบบเดิมตาม schema

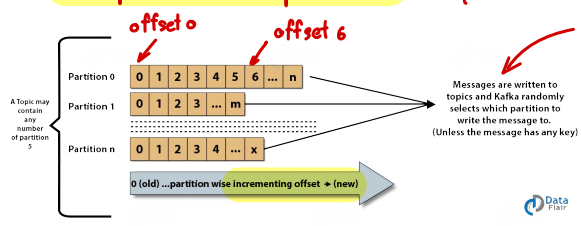
• Kafka → เป็น distributed streaming platform (คล้าย flume) มี 3 key ในการทำงาน



- 1.) Publish และ Subscribe ของ streaming record เป็นกรณีแบบ streaming data เมื่อนับเป็น message queue คือรับ data มา แล้วพักไว้ ได้เมื่อต้องการใช้
- 2.) มีระบบ fault-tolerant ของ streaming record
- 3.) สามารถ process stream record ได้ อย่างเช่น flume ที่ทำ ingestion ได้เป็นอย่างดี

• Concepts about Kafka

- kafka คือ **รันบนคลัสเตอร์ Cluster** โดยคลัสเตอร์ 1 server เรียกว่า **broker** (รันกับฝั่งรับ / ฝั่งส่ง)
- ตัว kafka cluster คือเก็บ **stream record** ใน categories เรียกว่า **topic** (เหมือนเป็น message queue)
- ในแต่ละ record คือประกอบด้วย **key/value** และ **timestamp**
- **Producer** คือเป็นคนส่ง publish message มาที่ topic
- **Consumers** คือเป็นคนดึง record ที่ถูก publish เข้ามาใน topic แล้วไป process ต่อ หรือ write ลง disk
- Broker และ consumer ติดต่อ **Zookeeper** เพื่อเก็บ state ของ broker ว่าทำงานอยู่ fail อยู่ และจัดการกับผู้ consumer ด้านข้อมูลไป แล้วทำด้าน tracking (track message offset)
- **1 topic แบ่ง 4 partitions (4 queue)** แต่ละ partition คือถูก map ด้วย log file



file คือ **Topic** ไปยัง **broker** ของ **producer** write ต่อท้าย

* **รับส่งแต่ละ = partition** ไม่ได้เรียงตาม **first in - first out**

- แต่ละ partition คือมีการ **replicate** ไว้บน server หลายตัว **fault-tolerance** และสามารถเก็บไว้ที่ server ได้
- **Producer** เป็นคนกำหนด ว่า write ลง partition ไหน
- consumer ต้องกำหนดว่าดึงข้อมูลอยู่ใน consumer group ไหน
- แต่ละ record ที่ publish เข้ามาใน topic คือถูกส่งไปให้ 1 consumer instance ที่อยู่ใน consumer group ที่ subscribe topic นั้น → **1 topic หลาย consumer group**
- ถ้า consumer instance อยู่ใน consumer group เดียวกัน คือมีการ **balance load** ในแต่ละ instance ใน group นั้น
- ถ้า consumer instance อยู่ต่าง consumer group กัน แต่ละ record คือถูก **broadcast** ไปหลาย group
- **Producer** และ **Consumer** คือมีการจับคู่กันไว้ด้วยระบบ เช่น 1-to-1, 1-to-many, many-to-1, many-to-many
- ถ้าใช้ kafka ในกรณี process ตาม queue producer จะส่งไปให้ partition เดียว ไม่รับหลาย partition
- process คือรับแล้ว ส่งไปหาคนที่ sequence มีคิวรอทำต่อไป

• Advantages of kafka

- โดยสรุปแล้ว kafka เป็นคนกลาง (messaging system) ในกรณีรับส่งข้อมูลระหว่างฝั่งส่ง (Producer) และฝั่งรับ (Subscriber)
- มี **Scalability** สามารถเพิ่มจำนวน consumer และ broker ได้ง่าย
- **Message durability** เพราะ support fault-tolerance

Streaming data